CacheLab 实验报告

姓名：匿名（公开仓库版）
学号：不公开

验证日期：2026-11-10

Part A /100	case1 加速比	case2 加速比	case3 加速比	综合加速比	Part B /100
100.00	9.62	9.45	7.24	**8.62**	100.00

主要结果。 Part A 官方 20 组测试全部通过；Part B 四个 case 均为 OK，综合加速比精确到四位小数为 **8.6176**，超过满性能分门槛 8。这里的两个百分制分数分别对应课程中的 40 分正确性和 40 分性能；报告与风格的 20 分仍由助教评阅。

主要方法。将一个输出子矩阵完整保存在寄存器中，沿归约维完成全部乘加后只写回一次；分别使用 6 × 4、4 × 4、4 × 6 的寄存器块。再通过逆序归约、蛇形遍历和块边界加载顺序，减少直接映射缓存的重复装入。算法仍执行通常的三重乘加，输入值均在运行时读取，不依赖样例数值，不使用 buffer。

环境为既有 QEMU 中的 Ubuntu 24.04.5 LTS、x86-64、GCC/G++ 13.3.0；使用原 Makefile 和官方评测脚本。实验中的 latency 是模拟访存代价，不是 QEMU 墙钟时间。所有最终结果来自同一版提交源代码。

本地评测证据

下图为通过本机 ttyd 真实终端连接既有 QEMU 后，分别运行 `python3 test/csim_test.py` 和 `python3 test/gemm_test.py` 所得的窗口截屏，包含所有 case、状态和分数。仅裁去终端右侧及底部空白，未修改评测内容；两个进程退出码均为 0。另已串行运行 `python3 test/overall_test.py` 复核，完整文本保存在 `report/evidence/pre-submission-audit-20261110.txt`。

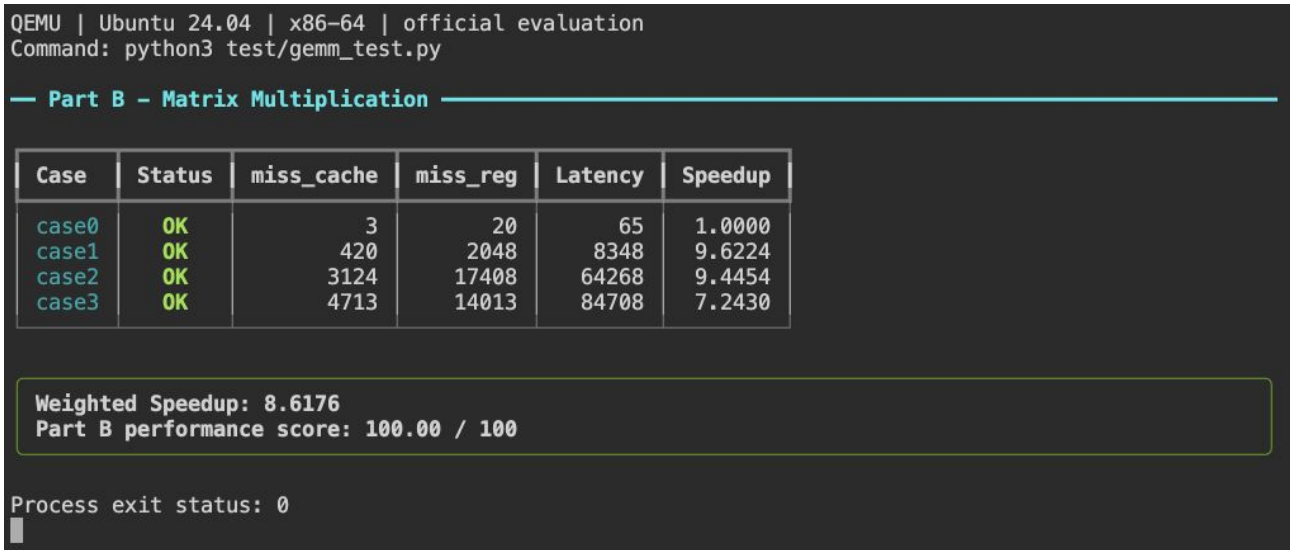


Figure 1: Part B：完整官方输出（真实终端截图）

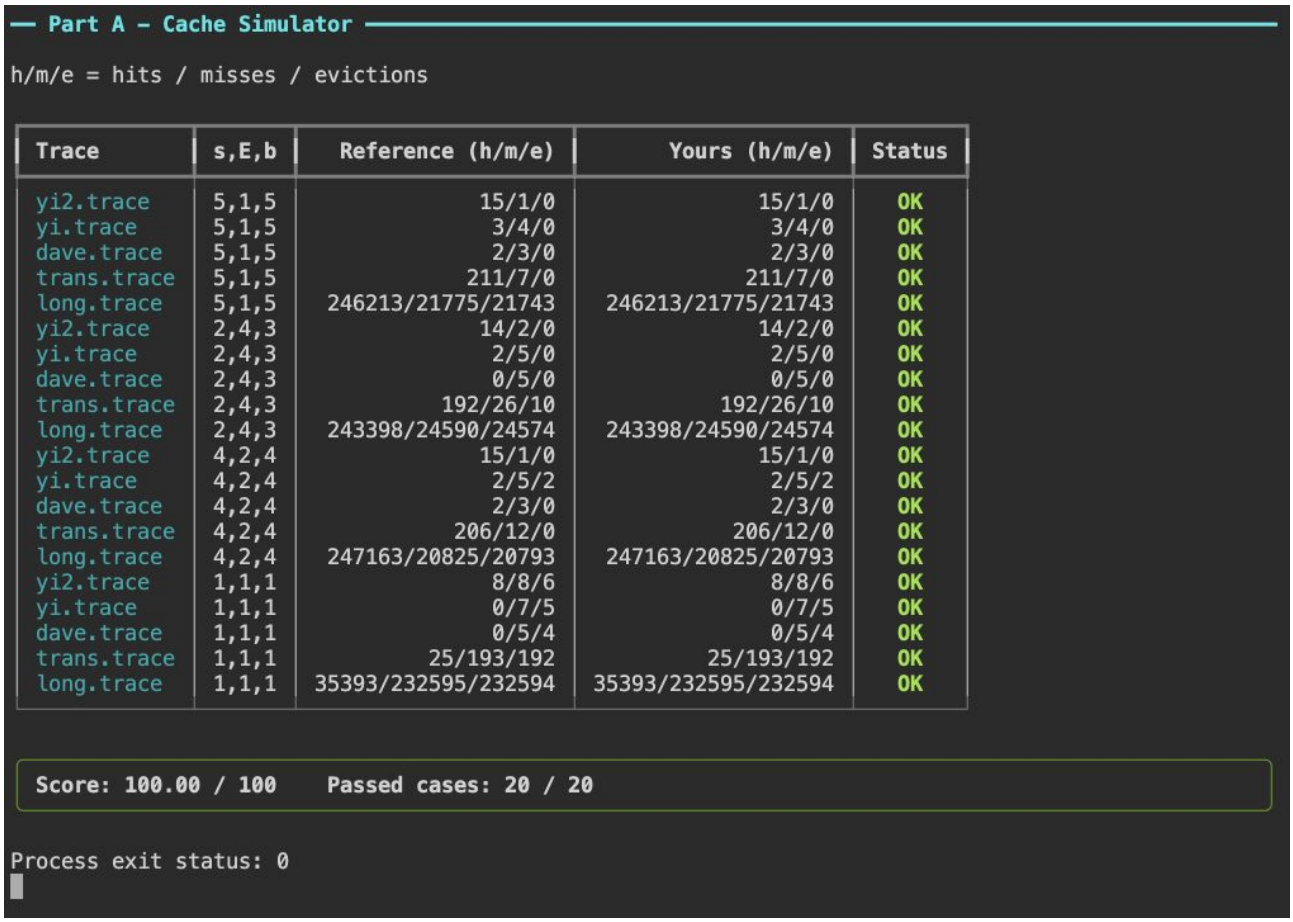


Figure 2: Part A：完整 20 组官方输出（真实终端截图）

Part A 的各行 hits / misses / evictions 均与官方参考程序一致。编译的基本合法性检查、矩阵结果、输入保持不变和寄存器上限检查也均通过。最终代码只改变 `csim.c` 和 `gemm.cpp`；Makefile、框架、基准、参考程序和官方测试文件保持原样。

Part A：cache 模拟器

实现与正确性

按组连续分配 `2^s × E` 个 cache line，每项保存 valid、tag、dirty 和最后访问的时间戳。地址用 `uint64_t` 解析，支持带或不带 `0x` 的十六进制输入。组号为 `(address >> b) & (2^s − 1)`，tag 为 `address >> (s+b)`；当 `s+b=64` 时直接令 tag 为 0，避免未定义移位。

每次访问只扫描目标组。命中时增加 hits 并更新时间戳；缺失时增加 misses，优先填空行，否则替换时间戳最小的有效行并增加 evictions。每次命中也更新次序，保证 LRU 而非 FIFO。S 与 L 都执行 write-allocate，S 设置 dirty；dirty 替换代表 write-back，但不额外增加一次 CPU 访问统计。题目只要求 h/m/e，不保存实际数据或统计回写流量。

时间复杂度为每条访问 O(E)，空间为 O(2^s E)。访问大小和寄存器编号不影响本题按起始地址统计的行为；额外兼容 I（忽略）和 M（读后写）。程序处理缺失参数、非法数值、打开文件失败和分配溢出；资源在正常与错误路径均释放。保留官方 `printSummary` 输出接口。

补充验证与参考程序边界

除官方 20 组外，新增独立测试覆盖 8 种配置：64 组低 32 位随机 trace 与参考程序对照，64 组完整 64 位 trace 与 Python `OrderedDict` LRU 模型对照；另测空 trace、M/I、非法参数及 `s+b=64`。

一个可复现边界是 `s=0,E=1,b=0`，依次读取地址 `0, 0x100000000, 0`。按题目 64 位地址约定应为 h/m/e = 0/3/2；提供的参考二进制实测为 2/1/0，会合并这两种地址。本实现保留完整 64 位语义，不跟随该别名行为；该差异不影响官方 20 组成绩。

Part B：共同设计

亮点与寄存器预算

1. **输出常驻寄存器**。累加器零初始化，完全不读取初始 C；每个输出只 store 一次，避免沿 k 分块造成的 C 反复读写。
2. **一个输入向量、多行或多列复用**。先取一侧向量，再逐个读取另一侧标量；同一输入参与多次乘加。内部展开固定小块，省去额外循环计数器。
3. **方向与生命周期共同设计**。不增加辅助方向变量，复用 i、j、k 的表达式控制蛇形和正反向；输入临时寄存器退出作用域后才开始输出循环，保持峰值不超过 36。

case	C 寄存器块	输入寄存器	指针与循环	峰值
case1	6 × 4 = 24	B 向量 4 + A 标量 1	指针 4 + i,j,k 3	36
case2	4 × 4 = 16	向量 4 + 标量 1	指针 4 + i,j,k 3	28
case3	4 × 6 = 24	A 向量 4 + B 标量 1	指针 4 + i,j,k 3	36

实测峰值与预算一致。边缘块只加载、写回有效位置；未使用的累加器虽存在但不产生越界访存。32 组输入模式 × 4 个尺寸另经 ASan/UBSan 验证，覆盖随机正负值、全零、全一、交替符号，以及非零初始 C；A、B 和未使用的 buffer 均保持不变。

性能与理论访存量

设 A 为 m 行 × n 列，B 为 n 行 × p 列，C 为 m 行 × p 列；寄存器输出块为 R 行 × Q 列。边缘裁剪后，实际访存次数恰为：

$$miss_{reg} = mn\lceil p/Q \rceil + np\lceil m/R \rceil + mp.$$

前三项分别是 A 加载、B 加载和 C 存储。计分代价为 `latency = 15 × miss_cache + miss_reg`。

case	原始 latency	最终 cache miss	最终 reg miss	最终 latency	加速比
case0（不计分）	65	3	20	65	1.00
case1	80328	420	2048	8348	9.62
case2	607040	3124	17408	64268	9.45
case3	613542	4713	14013	84708	7.24

综合加速比使用未四舍五入的值计算：`0.3 × 9.62242453 + 0.3 × 9.44544719 + 0.4 × 7.24302309 = 8.61757075`。

case1：16 行 × 16 列的方阵乘法

每次保存 6 行 × 4 列的 C，先加载连续四个 B 元素，再依次加载最多六个 A 元素。B 的每个 16 字节 cache line 恰好覆盖四列，在寄存器中复用六次。最后一个行块仅四行。理论 reg miss = 256 × 4 + 256 × 3 + 256 = **2048**。

A、B 各 64 条 cache line，相隔 64 条，映射到相同的 32 个组；交替逐元素加载容易冲突。先保留整个 B 向量避免四次乘加间反复重读 B。将 k 改为 15 至 0 后，跨输出块的缓存边界状态更有利，cache miss 从 434 降至 **420**。最终 B 加载恰为三轮完整矩阵的 3 × 64 = 192 次缺失，C 为 64 次，A 为 164 次。这个分解精确复现了 420，而不是把所有块都假设为冷缓存。

case2：32 行 × 32 列的方阵乘法

使用 4 行 × 4 列累加器。A 同列相邻行的组号相差 32/4 = 8，四行恰好分布于四个组；若继续增大行块，第五行就可能与第一行冲突。每轮 k 读取 A 四个值和 B 四个连续值，完成 16 次乘加；理论 reg miss = 1024 × 8 + 1024 × 8 + 1024 = **17408**。

行块间采用蛇形列遍历，使上一个行块末尾的 B 区域成为下一行块起点。`k % 4 == 0` 时先读取 B 向量，其余 k 先读取 A 向量；两分支临时变量互斥，峰值仍为 28。这一调整在当前映射下使 cache miss 从 3200 降至 **3124**。其中 B 为 8 × 256 = 2048 次，C 为 256 次，A 为 820 次；仍有容量与冲突损失，不能只用总数据大小估算性能。

case3：27 行 × 41 列乘以 41 行 × 27 列

选择 4 行 × 6 列输出块，装入 A 四值，再让每个 B 标量服务四个输出。末尾分别为三行和三列，所有边界均显式判断。理论 reg miss = 1107 × 5 + 1107 × 7 + 729 = **14013**。

列块按行块奇偶蛇形遍历；相邻块交替正反向遍历 k，减少从归约末尾突然跳回开头的距离。cache miss 从 5003 降至 **4713**。行跨度 41、27 均非四的倍数，B 起点也偏移三个 int，连续六列可能跨两到三条 cache line；C 列块边界可能拆开同一 line。因此 A/B/C 缺失分别为 1605/2826/282，不能照搬整齐方阵的 “一行块一条 line” 推导。此例的速度低于前两例，但最终顺序调整减少了 290 次缺失，收益最大。

渐进优化与排序

以下均为实际运行结果；首版使用相同寄存器块，但 i、j、k 全部递增，case2 总是先取 A 向量。最终版只改变遍历与加载顺序，不增加任何内存读写。

阶段	case1 加速比	case2 加速比	case3 加速比	综合加速比
原始三重循环	1.00	1.00	1.00	1.00
寄存器分块，正向遍历	9.39	9.28	6.89	8.36
最终访存顺序	**9.62**	**9.45**	**7.24**	**8.62**

最优秀的实现排序：**case3、case1、case2**。case3 同时处理不规则尺寸、满额寄存器和方向复用；case1 以直接的向量复用取得最高加速比；case2 保留 8 个空闲寄存器，因为盲目增大块会破坏缓存映射，寄存器占满不等于访存最优。

缺失分析的复现

先在仓库根目录执行注释中的 Shell 命令生成三个 trace，再将下面代码保存为 `replay.py`，设置 `case` 为 1、2、3 后运行 `python3 replay.py`。输出依次为 A、B、C 的 [访问数, 缺失数]；这是 trace 回放计数，而非解析公式。

```
# Shell: make printTrace && for c in 1 2 3; do ./printTrace case$c > case$c.trace; done
case = 3
m, n, p = {1:(16,16,16), 2:(32,32,32), 3:(27,41,27)}[case]
bounds = [m*n, m*n+n*p, m*n+n*p+m*p]
tags, result = [None]*32, [[0,0] for _ in range(3)]
with open(f"case{case}.trace") as trace:
    for line in trace:
        word = (int(line.split()[1].split(',')[0], 16) - 0x30000000) // 4
        owner = next(i for i, end in enumerate(bounds) if word < end)
        block, index = word // 4, (word // 4) % 32
        result[owner][0] += 1
        if tags[index] != block:
            result[owner][1] += 1
            tags[index] = block
print(result)
```

只按冷缺失下界估算会得到 case1/2/3 分别为 192/768/736 条 line；实际还有跨块重载和组冲突。以上回放将精确计数与机制分析分开，避免把碰巧吻合的近似当成证明。

收获与局限

本次优化最关键的取舍是同时控制两个层级：增大寄存器块可以减少 reg miss，却也可能增加同组冲突；沿 k 分块或搬运到 buffer 会额外引入读写，必须纳入 `15M+R` 后比较。最终选择保持 C 常驻寄存器的简单方案。方向优化仅适用于本题固定尺寸和映射，不能据此声称通用 GEMM 或真实 CPU 同样加速 8.62 倍，也没有证明全局最优。

参考的重要资料

- 课程实验说明及优化提示：计分模型、LRU、write-allocate、寄存器复用和约束。
- 官方 demo.cpp：允许的 `reg` / `ptr_reg` 操作；报告要求。
- 实现、参数比较、独立验证和报告整理使用了 Codex 辅助；未引用外部学生解答。所有性能结论均以未修改的官方框架实测为依据。